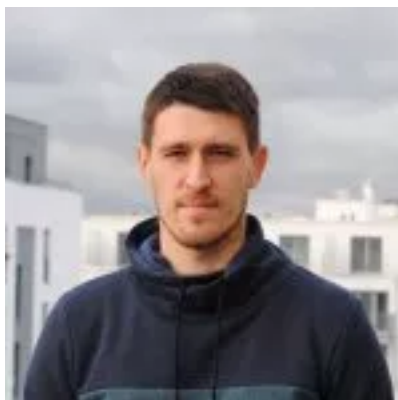


Language



Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



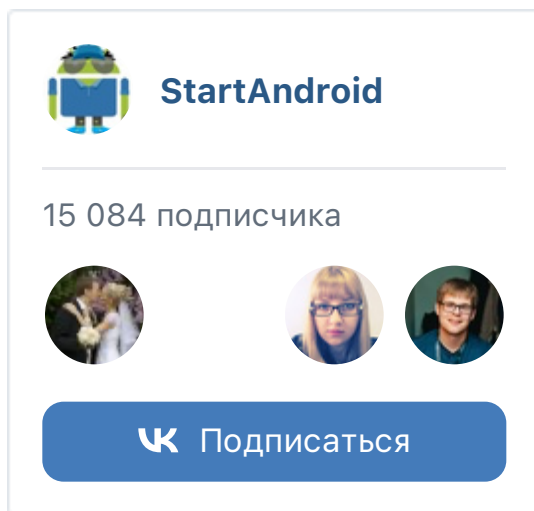
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

[Яндекс](#)
410011180491924

[WebMoney](#)
R248743991365
Z551306702056

[Paypal](#)

Урок 2. Граф зависимостей

21 июня 2021

В этом уроке узнаем, что такое граф компонентов и научим даггер создавать объекты с не пустым конструктором.

Совокупность всех объектов, которые умеет создавать компонент - это граф объектов компонента или граф зависимостей компонента.

В примерах прошлого урока граф состоял всего из двух объектов: DatabaseHelper и NetworkUtils. Эти объекты имели пустые конструкторы и легко создавались. Но чаще мы имеем дело с более сложными объектами, у которых в конструктор должны передаваться другие объекты. Т.е. компоненту для создания одного объекта может потребоваться создать другой объект. Давайте посмотрим, как это сделать в даггере.

Будем продолжать работать с примером из прошлого урока. Мы там уже создали компонент и модули, благодаря чему даггер умеет создавать объекты DatabaseHelper и NetworkUtils.

Добавим в проект новый класс:

```
1 class MainActivityPresenter {
2     private val databaseHelper: DatabaseHelper,
3     private val networkUtils: NetworkUtils
4 } {
5
6     // ...
7
8 }
```

Это презентер для MainActivity. Для создания ему требуются DatabaseHelper и NetworkUtils. Давайте научим даггер создавать такой презентер.

Создаем модуль, а в нем Provides метод, который будет создавать объект MainActivityPresenter:

```
1 @Module
2 class MainModule {
3
4     @Provides
5     fun provideMainActivityPresenter(): MainActivityPresenter {
6         return MainActivityPresenter()
7     }
8 }
```

Такой код у нас не сработает. Потому что у MainActivityPresenter нет пустого конструктора. Ему нужны DatabaseHelper и NetworkUtils.

Где их взять? Просто добавить как аргументы метода и использовать их в конструкторе:

```
1 @Provides
2 fun provideMainActivityPresenter(databaseHelper: DatabaseHelper, networkUtils: NetworkUtils): MainActivityPresenter {
3     return MainActivityPresenter(databaseHelper, networkUtils)
4 }
```

Когда мы попросим даггер создать презентер, он обратится к методу provideMainActivityPresenter и увидит, что ему на вход требуются объекты DatabaseHelper и NetworkUtils. И т.к. даггер уже умеет создавать эти объекты, то он просто создаст их и передаст в этот метод и получит созданный презентер.

Т.е. даггер создает объекты не только для того, чтобы вернуть их нам или заинжектить куда-либо, но и для своих внутренних целей, если это потребуется.

Теперь даггер знает, как создавать объект MainActivityPresenter. И мы можем попросить этот объект у компонента:

```
1 @Component(modules = [StorageModule::class, NetworkModule::class])
2 interface AppComponent {
3     fun getMainActivityPresenter(): MainActivityPresenter
4 }
```

Создаем get метод, который будет возвращать презентер.

Все вроде сделано правильно, но при попытке запустить проект мы получим ошибку:

MainActivityPresenter cannot be provided without an @Inject constructor or an @Provides-annotated method

Я специально сразу ее показал, потому что вы периодически будете встречать ее в работе. И будет полезно, если вы сразу ее поймете. Этой ошибкой даггер сообщает нам, что не может создать объект MainActivityPresenter, потому что не знает как. Он говорит, что не нашел ни Inject конструктора, ни метода с аннотацией Provides.

Inject конструктор - это альтернативный способ создания объектов. О нем мы еще поговорим в одном из следующих уроков.

А вот метод с аннотацией Provide мы точно создавали. Это был метод provideMainActivityPresenter в модуле MainModule. Почему даггер говорит, что его нет? Потому что модуль то мы создали, но компоненту об этом не рассказали. Мы должны явно указать его в @Component(modules = []). Только тогда компонент сможет использовать методы этого модуля для создания объектов. И он увидит, что там есть Provides метод для создания объект MainActivityPresenter.

Добавим MainModule модуль в список модулей в аннотации Component:

```
1 @Component(modules = [StorageModule::class, NetworkModule::class, MainModule::class])
2 interface AppComponent {
3     fun getMainActivityPresenter(): MainActivityPresenter
4 }
```

После этого мы можем просить презентер у компонента в MainActivity:

```
1 class MainActivity : AppCompatActivity() {
2
3     lateinit var mainActivityPresenter: MainActivityPresenter
4
5     override fun onCreate(savedInstanceState: Bundle?) {
6         super.onCreate(savedInstanceState)
7         setContentView(R.layout.activity_main)
8
9         mainActivityPresenter = (application as App).appComponent.getMainActivityPresenter()
10    }
11 }
```

Граф зависимостей компонента теперь включает три объекта: MainActivityPresenter, DatabaseHelper, NetworkUtils. Причем нам нужен только один из них - презентер. Два других мы напрямую у компонента не запрашиваем, но он сам создает их, чтобы создать презентер.

Давайте чуть усложним пример. Сделаем так, чтобы объекты DatabaseHelper и NetworkUtils сами в свою очередь были составными и требовали другие объекты при своем создании.

Например, так:

```
1 class DatabaseHelper(private val repository: Repository)
2
3 class NetworkUtils(private val connectionManager: ConnectionManager)
```

Чтобы это заработало нам нужно будет сделать следующее:

1) В модулях создать Provides методы для создания объектов Repository и ConnectionManager

```
1 @Provides
2 fun provideRepository(): Repository {
3     return Repository()
4 }
```

```
1 @Provides
2 fun provideConnectionManager(): ConnectionManager {
3     return ConnectionManager()
4 }
```

Это можно сделать в уже существующих модулях StorageModule и NetworkModule, они вполне подходят.

2) Добавить Repository и ConnectionManager как аргументы в Provides методы создания DatabaseHelper и NetworkUtils:

```
1 @Provides
2 fun provideDatabaseHelper(repository: Repository): DatabaseHelper {
3     return DatabaseHelper(repository)
4 }
```

```
1 @Provides
2 fun provideNetworkUtils(connectionManager: ConnectionManager): NetworkUtils {
3     return NetworkUtils(connectionManager)
4 }
```

И использовать их в конструкторах.

3) Если создавали какие-то новые модули, то добавить их в Component(modules = [])

Теперь граф зависимостей даггера содержит 5 объектов. И только один из них мы явно запрашиваем. А остальные 4 создаются даггером автоматически.

Ошибки

Иногда даггер показывает ошибки, даже если все сделано правильно. Проблема обычно в кэшировании. Он себе там что-то закэшировал и не подтягивает новые изменения.

В этом случае есть пара действий, которые могут помочь:

- 1) Build -> Clean project
- 2) File -> Invalidate cache and restart -> Invalidate and restart

Но иногда не помогает даже это. Тогда попробуйте переименовать объект, на который даггер ругается. Просто поменяйте одну букву в имени с помощью SHIFT+F6. И попробуйте запустить проект. Если все ок, то возвращайте имя обратно, ошибка не должна вернуться.

Ну а если ничего из этого не помогает, то возможно все таки что-то сделано неправильно и даггер ругается не просто так.

Назад

Вперёд

Добавить комментарий

Тема

😄 😊 😌 😍 😎 😏 😐 😑 😒 😓 😔 😕 😖 😗 😘 😙 😚 😛 😜 😝 😞 😟 😠 😡 😢 😣 😤 😥 😦 😧 😨 😩 😪 😫 😬 😭 😮 😯 😰 😱 😲 😳 😴 😵 😶 😷 😸 😹 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿

Отправить

3Comments

